

CME 213 Final Project

Building and Profiling a Small GPT-2 Training Pipeline in CUDA and MPI

Nils Astrup Toft

Sondre Rogde

June 8, 2026

Abstract

We build a GPT-2-style transformer training pipeline from scratch in CUDA and MPI, distributed across four GPUs on a single node. We use Nsight Compute and Nsight Systems to identify what limits performance: matrix multiply sitting far below compute peak, memory-bound kernels under-utilizing bandwidth, redundant MPI synchronization overhead, and dynamic GPU allocations stalling the backward pass. We address each bottleneck in turn, with register-tiled GEMM, vectorized memory loads, a fused gradient allreduce, and pre-allocated backward scratch buffers. Together these cut 4-GPU step time by roughly 20% and raise strong-scaling efficiency from 55% to 62%. For inference, we implement a PagedAttention decode kernel that more than doubles KV-cache memory utilization at large batch sizes with negligible latency overhead.

1 Project Description

We train a decoder-only GPT-2-style transformer [9] on sequences of token IDs. We focus on the fundamental kernels in the transformer architecture and how we can optimize these. We use synthetic random tokens for scaling and profiling experiments and Wikitext-2 for realistic training dynamics. However, the main focus for the project is profiling and optimization analysis, not a competitive LLM. A single forward step has 36+ GEMMs and $O(S^2)$ attention in naive form. Realistic batch sizes easily outgrow single-GPU memory and per-step latency multiplies into hours at full scale. This motivates both multi-GPU training and KV-cache management for inference.

Setup. Inputs: a batch of B integer sequences of length S . Outputs: per-position vocabulary logits, reduced to a scalar cross-entropy loss and backpropagated through every parameter with Adam [6]. Each Pre-LayerNorm block applies multi-head causal self-attention then a two-layer GELU FFN, both with residuals. Notation: B batch, S sequence, C model dim, H heads, $D=C/H$, V vocab. All tensors FP32 row-major. We use the class compute cluster which has peak FP32 16.31 TFLOPS; peak HBM 672 GB/s; ridge ≈ 24 FLOP/byte. Default config: $n_{\text{layers}}=6$, $C=256$, $H=8$ ($D=32$), $S=128$, $V=512$, which gives about 5 M parameters, 20 MB of gradient.

2 Algorithms and State of the Art

The below paragraphs discuss some of the algorithms and kernels we implemented. An overview of all implemented kernels can be found in the Appendix in Table 2.

Dense GEMM. cuBLAS [2] reaches 70–80% of FP32 peak via shared-memory tiling, register-file accumulation, and (Ampere+) tensor cores. We implement the shared-memory tiled variant from Boehm [1] with 16×16 thread-block register tiling. This is a slightly more optimized version compared to the one we implemented in class as it uses a more aggressive register tiling. Turing tensor cores only support FP16/INT8 [8], so we keep everything as FP32.

Attention. Flash Attention [3] tiles Q, K, V in SRAM with an online softmax, reducing memory from $O(S^2)$ to $O(S \cdot D)$ at the same output. We implement FA1 forward, the backward uses the naive $O(S^2)$ materialization. FA2’s recomputation backward [4] would be needed for longer sequences.

The full transformer. We implement Pre-LayerNorm transformer blocks as in GPT-2 [9], with Adam [6] as the optimizer.

Distributed training and inference. The standard approach is to replicate the model on each rank, have each rank compute gradients on a local mini-batch, and synchronise via `MPI.Allreduce` before the optimizer step. The naive implementation issues one `MPI.Allreduce` per parameter tensor (≈ 50 calls in our model) each carrying its own MPI startup latency and a PCIe device-to-host round trip. Horovod [10] and Karpathy’s `llm.c` [5] instead pack all gradients into a single contiguous buffer and issue one call. We implement the same strategy. More sophisticated methods with tensor parallelism (Megatron-LM [11]) can be used when models exceed single-GPU memory. We do not employ this as our 5 M-parameter model fits trivially on 24 GB. For inference, vLLM’s PagedAttention [7] manages variable-length KV caches via virtual-memory-style block tables. We implement this for the decode step as discussed in the last guest lecture.

3 Parallelization Strategy

This section presents the final optimized design. The bottlenecks that motivated each decision and the variants explored to reach it are covered in Section 6.

System overview. We organize our system in five layers. CUDA kernels, stateful modules (Linear, LayerNormLayer, EmbeddingLayer, MultiHeadAttention, TransformerBlock), GPT2 model, `Trainer` (forward $\rightarrow$ CE loss $\rightarrow$ backward $\rightarrow$ Adam), and MPI wrapper.

GEMM (compute-bound). We implement three GEMM-variants (AB , AB^T , $A^T B$). Each uses a 128×128 output tile in shared memory with a $BK=8$ strip across K . The thread block is $16 \times 16 = 256$ threads, each accumulating an 8×8 register sub-tile (64 FP32 accumulators per thread). The inner k -loop is `#pragma unroll`, exposing 64 independent FMAs per smem load, raising both arithmetic intensity and occupancy. Global loads are coalesced. Shared-memory tiles are declared `[TILE][TILE+1]`. The $+1$ padding breaks the stride that would serialize warp writes onto a single shared-memory bank, eliminating bank conflicts.

Flash Attention and bandwidth-bound kernels. Flash Attention tiles Q, K, V into SRAM, maintaining a running max $m^{(j)}$ and normalizer $\ell^{(j)}$ with online correction $e^{m^{(j-1)} - m^{(j)}}$ (similar to what we did in the homework). Causal masking is per-tile, one block per (batch, head, query-tile). LayerNorm uses a two-pass block reduction. The two passes were necessary to avoid catastrophic cancellation when computing the variance. GELU, Softmax, Cross-Entropy, the LayerNorm output pass, and the residual `add_inplace` use `float4` 128-bit loads, which gives 4 floats per memory instruction, using the full 128-bit memory bus. Adam is a pure read-modify-write of ≈ 20 bytes per parameter and is bandwidth-dominated by construction.

MPI data parallelism. Each rank owns a full model copy on one GPU. We use two collectives: `MPI.Bcast` once at startup to synchronize initial weights, and `MPI.Allreduce(MPI_SUM)` after each backward pass to average gradients across ranks. Batch scattering requires no MPI call. All ranks

generate the same random shuffle from a shared step seed and each independently selects its own slice, so no data moves over the network. Because all ranks then run an identical Adam update on the same averaged gradient, weights stay synchronized without a further broadcast.

Non-obvious implementation challenges. Two issues required profiler-assisted debugging rather than correctness tests alone. The +1 shared-memory tile padding was revealed by Nsight Compute’s bank-conflict counters. A unit test passes either way. The second was the per-step `cudaMalloc` stalls in the backward pass. This was only revealed by the Nsight Systems timeline and is further discussed in Section 6.

4 Performance Model

We use three complementary models to predict how our implementation performs, one per regime.

The Roofline model [12] measures whether a kernel is memory- or compute- bound. Achievable performance is $\min(\text{peak FLOPS}, B \cdot \text{intensity})$. Basic tiled GEMM (one output per thread, $T=32$) has intensity $T/4=8$ FLOP/byte, sitting below the ridge and shared-memory-bandwidth-bound. With 8×8 register tiling, intensity rises to 64 FLOP/byte, above the ridge. *Prediction:* register tiling moves GEMM from memory- to compute-bound. LayerNorm, GELU, Softmax, and Cross-Entropy have $O(1)$ intensity and stay memory-bound. These kernels therefore benefit from more efficient memory management. So with vectorized loads using `float4` we should get roughly a $4\times$ better HBM utilization. The decode kernel sits at ≈ 0.5 FLOP/byte, far below the ridge. With one block per (request, head), the grid has $H \cdot B$ blocks. For $H=8$ the grid fills 72 SMs only at $B \geq 9$, so small batches are SM-count-limited before bandwidth even becomes relevant.

Communication and scaling models. We model allreduce latency as $T(n) = \alpha + \beta n$, where α captures per-call startup and $\beta = 1/b$ is inverse bandwidth. For our 20 MB gradient, the bandwidth term predicts ≈ 7 ms per PCIe transfer direction. Because our implementation is not CUDA-aware, each call incurs a D→H copy and an H→D copy, adding $\approx 2n/b_{\text{PCIe}} \approx 13$ ms on top of the network cost. The gap between the fitted curve and the ideal network bandwidth is therefore this D→H→D round-trip cost, recoverable only with CUDA-aware MPI. When we designed the fused-allreduce variant, we expected to recover most of the ≈ 18.6 ms overhead by eliminating per-call startup. At the time we failed to account for this PCIe round-trip cost, which dominates regardless of call count.

Amdahl and isoefficiency. For strong scaling with $T_P = T_{\text{compute}}/P + T_{\text{comm}}$, parallel efficiency is $E(P) = T_1/(PT_P)$. Because T_{comm} is fixed by gradient size while T_{compute} shrinks with P , efficiency degrades as more GPUs are added. For isoefficiency, maintaining a fixed efficiency as P grows, the total batch must scale proportionally with P .

5 Benchmarking and Instrumentation

Tools. We use Nsight Compute to capture per-kernel TFLOPS, GB/s, occupancy, and roofline coordinates, and Nsight Systems to profile the full training timeline. Both runs capture all kernels over 20 training steps. This allows us to compare measurements with our performance models above.

Standalone benchmarks. `allreduce_bench` fits $\alpha + \beta n$ for $P \in \{2, 4\}$ across 256 B–64 MB message sizes (100-trial averages). `paged_attention_bench` measures decode latency and KV-cache memory utilization across a sweep of batch sizes.

We defer the results and comparison to the performance model to Sections 6–7.

6 Bottleneck Analysis and Algorithmic Variants

We consider the implementation that led to the parallelization strategy explained above. First we identify four training bottlenecks and a fifth inference regime using Nsight Compute and Nsight Systems. We then use these empirical observations to optimize the implementation. We consider 5 different algorithmic variants. Figure 1 shows the kernels before and after our optimizations.

Register tiling. As predicted GEMM is initially below the roofline ridge. Without register tiling, Nsight Compute places GEMM at 2.20 TFLOPS = 13.5% of FP32 peak and 8 FLOP/byte, left of the ≈ 24 FLOP/byte ridge (Fig. 1). The kernel is memory-bound in shared memory, not global memory. Each thread reads shared-memory tiles without accumulating enough FMAs per read to saturate the FP32 units.

We remedy this by adding register tiling. We extend one-output-per-thread to an 8×8 register sub-tile. This raises arithmetic intensity from 8 to 64 FLOP/byte, which crosses the hardware’s ridge point of ≈ 24 FLOP/byte and shifts the bottleneck from shared-memory bandwidth to compute. Nsight Compute confirms that throughput rises from 2.20 to 9.56 TFLOPS (13.5% $\rightarrow$ 58.6% of FP32 peak, $4.3\times$). We benchmark against cuBLAS which reaches 70–80%. We attain $\sim 75\%$ of that (Fig. 1, Table 3).

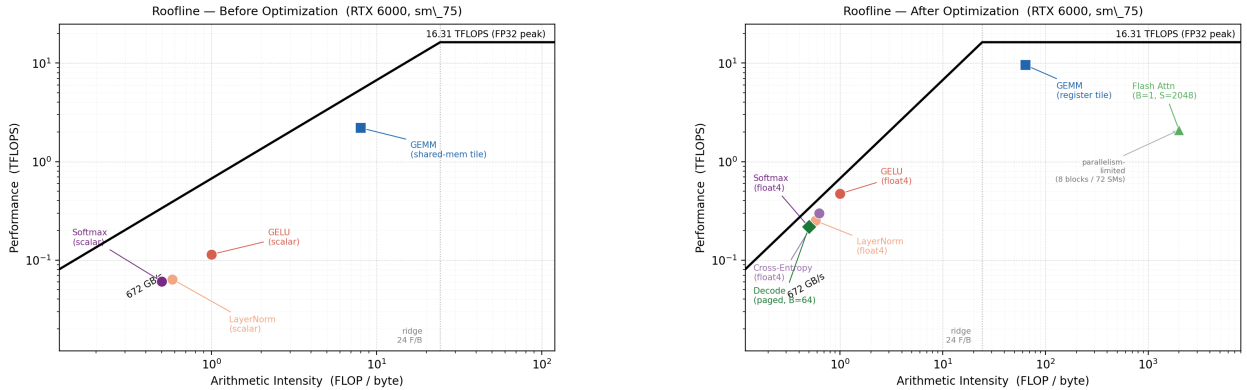


Figure 1: Roofline analysis. Left: before optimization, Right: after optimization.

Scalar loads to vectorized float4 loads. We improve the memory-management of the $O(1)$ kernels. Scalar 32-bit loads in GELU, Softmax, Cross-Entropy, LayerNorm, and `add_inplace` are replaced by `float4`, issuing one 128-bit transaction per four floats. Nsight Compute confirms the gain and bandwidth rises to 430–471 GB/s (64–70% of HBM peak) across all five kernels, versus scalar baselines of ≈ 16 –18%. This is a $\approx 4\times$ improvement (Table 3) as expected.

Per-tensor allreduce to fused gradient allreduce. Our first implementation had 50 separate `MPIAllreduce` calls per step each triggering a $D \rightarrow H$ `cudaMemcpy`, a collective, and an $H \rightarrow D$ `cudaMemcpy`. The 50 calls cumulate to ≈ 18.6 ms/step which we want to reduce.

We therefore replace the 50 individual `MPI_Allreduce` calls with one contiguous 20 MB pinned buffer to eliminate redundant startup overhead. Fitting $\alpha + \beta n$ to the `allreduce_bench` data gives $\alpha = 51.2 \mu\text{s}$ for $P=2$, so eliminating 49 extra call startups saves $49 \times 51.2 \mu\text{s} \approx 2.5 \text{ ms}$, consistent with the measured $\approx 3 \text{ ms}$ gain. The saving is smaller than what we predicted because fusion does not reduce the total bytes crossing PCIe. The full 20 MB gradient still traverses $D \rightarrow H \rightarrow D$ in both cases. At $\approx 3 \text{ GB/s}$ effective PCIe staging bandwidth (Fig. 4) that round trip costs $\approx 13 \text{ ms}$ regardless of call count and cannot be recovered without CUDA-aware MPI to route gradients directly over NVLink.

From per-step `cudaMalloc` in backward to pre-allocated backward scratch. The Nsight Systems timeline (Fig. 2) shows $\approx 30 \text{ ms}$ of idle GPU time per step as `cudaMalloc` barriers drain the kernel queue. The multi head attention and transformer block allocated 18 temporary buffers in each call, including two $B \cdot H \cdot S^2$ attention-score matrices ($\approx 4 \text{ MB}$ each at $B=16$).

The eighteen per-step buffers are moved from the backward call to the constructor, eliminating the $\approx 30 \text{ ms}$ of per-step `cudaMalloc` barriers. Figs. 2 and 3 show the Nsight Systems timeline before and after. The allocations and the resulting idle GPU time are absent in the optimized profile.

PagedAttention for inference. With one block per (request, head), the grid has only $H \cdot B$ blocks. At $B=1$ this gives 8 blocks for 72 SMs, leaving most of the device idle. At larger B the grid fills the device and the kernel transitions to being bandwidth-limited, consistent with its $\approx 0.5 \text{ FLOP/byte}$ arithmetic intensity.

We improve the memory-management in inference with PagedAttention. Block-table indirection raises KV-cache utilization from 39.4% to 88.0% at $B=64$ ($2.2\times$) with only 1.3% latency overhead, allowing the same memory budget to serve $2.2\times$ more concurrent requests (Table 4).

7 Scalability Analysis

Now we consider the third regime for modeling performance by looking at strong and weak scaling. Table 1 (Figs also included in Appendix E) reports strong and weak scaling before and after applying the four variants involved in training, as described above. At $P=1$ there is no allreduce, so Variant 3 contributes zero. The 14 ms reduction from 162.7 to 148.8 ms reflects Variants 1, 2, and 4 combined. At $P=4$, all four variants together drop step time from 74.5 to 59.8 ms. Communication falls $18.6 \rightarrow 15.6 \text{ ms}$ from fusing (V3) and compute falls $55.9 \rightarrow 44.2 \text{ ms}$ from kernel and allocation optimizations (V1+V2+V4), raising strong-scaling efficiency from 55% to 62%.

Config	Sweep	P	step (ms)	comm (ms)	comm%	speedup	eff.
Before opt.	strong	1	162.7	0.0	—	1.00×	100%
Before opt.	strong	2	105.1	17.7	16.8	1.55×	77%
Before opt.	strong	4	74.5	18.6	25.0	2.19×	55%
After opt.	strong	1	148.8	0.0	—	1.00×	100%
After opt.	strong	2	92.5	14.4	15.6	1.61×	80%
After opt.	strong	4	59.8	15.6	26.1	2.49×	62%
After opt.	weak	1	43.6	0.0	—	—	100%
After opt.	weak	2	57.9	14.3	24.6	—	75%
After opt.	weak	4	59.8	15.6	26.1	—	73%

Table 1: Scaling on one 4-GPU node (RTX 6000, sm.75). Strong: total $B=16$. Weak: local $B=4$ per rank. $n_{\text{layers}}=6$, $C=256$, $S=128$, 20-step averages.

Amdahl and isoefficiency. At $P=4$ after optimization, 44.2 ms is compute and 15.6 ms is allreduce (26% of step time), giving $E_4 = T_1/(4T_4) = 148.8/(4 \times 59.8) \approx 62\%$. We get $T_{\text{compute}}/T_{\text{comm}} = 44/15.6 \approx 2.8$. We can increase this ratio in two ways. First, we may increase the local batch (more compute per allreduce call), or second, we can adopt CUDA-aware MPI to cut comm to ~ 7 ms (removing PCIe staging), giving ratio $44/7 \approx 6.3$.

Weak scaling. Efficiency at fixed $B_{\text{local}}=4$ is 75% ($P=2$) and 73% ($P=4$) (see Fig. 5). Efficiency is below 100% because at $P=1$ there is no allreduce, whereas for $P>1$ every rank pays a fixed ≈ 15.6 ms allreduce overhead that does not shrink with local batch size, it is determined by the size of the gradient. Note that at $P=2$, weak efficiency (75%) is lower than strong (80%) because strong scaling at $P=2$ gives each rank $B_{\text{local}}=8$, doubling the compute per allreduce call. At $P=4$ both regimes coincide at $B_{\text{local}}=4$ and the weak scaling is better.

8 Correctness and Verification

Per-kernel. Every CUDA kernel is validated against a plain C++ CPU implementation with tolerances $\text{rel} \leq 10^{-3}$ or $\text{abs} \leq 10^{-5}$. The absolute guard prevents near-zero values from spuriously failing. Forward kernels are additionally compared to PyTorch references. All forward and backward tests pass. Table 5 reports the achieved errors for each kernel. The PagedAttention decode kernel is verified against the contiguous reference bit-identically ($\text{abs} = 0$, $\text{rel} = 0$) across all seven test configurations, including scrambled block-table orderings that would fail silently if the indirection were ignored. Both also match a CPU oracle within FP32 `expf` tolerance.

Distributed training. We perform two multi-rank consistency checks. The step-1 loss is bit-identical across $P \in \{1, 2, 4\}$ (6.5336), confirming that `MPI_Bcast` and the deterministic forward pass are correct. All configurations drive loss toward the random-token entropy floor. On Wikitext-2, the model trains correctly, falling from 9.27 to below 6.0 cross-entropy over 300 steps at $P=4$.

9 Discussion

Biggest payoffs. All four optimizations together cut 4-GPU step time by ≈ 15 ms (74.5 $\rightarrow$ 59.8 ms, $\approx 20\%$). Register tiling (4.3 $\times$ on GEMM throughput) and `float4` (64–70% of HBM peak) were identified and quantified by Nsight Compute. The allreduce fusion and scratch pre-allocation were only visible through Nsight Systems timeline profiling. The former as PCIe staging between the GPU and host, the latter as `cudaMalloc` barriers stalling the backward timeline. Of the four, fused allreduce delivered the least. It saved only ≈ 3 ms of the anticipated 18.6 ms and it was not worth it to implement it without CUDA-aware MPI.

Limitations and future work. The largest remaining lever is CUDA-aware MPI. The ≈ 8 ms of D $\rightarrow$ H $\rightarrow$ D PCIe staging per step would disappear over NVLink. Flash Attention backward is $O(S^2)$ and FA2’s recomputation backward [4] would be needed for longer sequences. BF16 [8] is not implemented because Turing has no BF16 tensor cores, so BF16 arithmetic runs on regular CUDA cores at FP32 throughput. On Ampere/Hopper, native BF16 tensor cores would roughly halve memory footprint and double matrix-multiply throughput relative to FP32. Finally, our 5 M-parameter model on Wikitext-2 reaches a perplexity floor that is too high for coherent text generation. Scaling to a full GPT-2 configuration (124 M parameters) on a larger corpus may give meaningful inference.

References

- [1] S. Boehm. How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance. *siboehm.com*, 2022.
- [2] NVIDIA. cuBLAS Library User Guide. NVIDIA Developer Documentation, 2024.
- [3] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.
- [4] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism. *arXiv:2307.08691*, 2023.
- [5] A. Karpathy. llm.c. <https://github.com/karpathy/llm.c>, 2024.
- [6] D. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. In *ICLR*, 2015.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [8] P. Micikevicius et al. Mixed Precision Training. In *ICLR*, 2018.
- [9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI Technical Report, 2019.
- [10] A. Sergeev and M. Del Balso. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv:1802.05799*, 2018.
- [11] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv:1909.08053*, 2019.
- [12] S. Williams, A. Waterman, and D. Patterson. Roofline: An Insightful Visual Performance Model. *CACM*, 52(4):65–76, 2009.

A All Implemented Kernels

Table 2 shows an overview of every kernel we implemented.

Table 2: Custom CUDA kernels implemented in this project.

Kernel	Operation	Pass
<i>Embedding & loss</i>		
<code>embedding_forward_kernel</code>	Token lookup	Fwd
<code>cross_entropy_forward_kernel</code>	Softmax + CE loss	Fwd
<code>cross_entropy_backward_kernel</code>	CE gradient	Bwd
<i>Linear layers (GEMM)</i>		
<code>gemm_nt_reg_kernel</code>	$Y = XW^T$	Fwd
<code>gemm_register_tiled_kernel</code>	$dX = dY W$	Bwd
<code>gemm_tn_reg_kernel</code>	$dW = X^T dY$	Bwd
<i>Normalization</i>		
<code>layernorm_forward_kernel</code>	LayerNorm	Fwd
<code>layernorm_backward_kernel</code>	LayerNorm gradients	Bwd
<i>Activation</i>		
<code>gelu_forward_kernel</code>	GELU activation	Fwd
<code>gelu_backward_kernel</code>	GELU gradient	Bwd
<i>Attention</i>		
<code>flash_attention_forward_kernel</code>	Flash attention (tiled)	Fwd
<code>softmax_backward_kernel</code>	Attention softmax bwd	Bwd
<code>attention_weights_kernel</code>	dQ, dK, dV gradients	Bwd
<i>Utility</i>		
<code>flat_to_bhsd_kernel</code>	Reshape to $[B, H, S, D]$	Fwd/Bwd
<code>bhsd_to_flat_kernel</code>	Reshape to $[BS, H, D]$	Fwd/Bwd
<code>add_inplace_kernel</code>	Residual addition	Fwd/Bwd
<i>Optimizer & inference</i>		
<code>adam_update_kernel</code>	Adam optimizer step	Opt.
<code>paged_attention_forward_kernel</code>	Paged KV-cache attention	Infer.

B Kernel Performance Details

Kernel	Bound	Baseline	% peak	Optimized	% peak
GEMM	Compute	2.20 TFLOPS	13.5%	9.56 TFLOPS	58.6%
Flash Attn	Compute	1.63 TFLOPS	10%	2.09 TFLOPS	12.8%
LayerNorm	BW	≈ 110 GB/s	16%	432 GB/s	64%
GELU	BW	≈ 114 GB/s	17%	471 GB/s	70%
Softmax	BW	≈ 121 GB/s	18%	431 GB/s	64%
Cross-Entropy	BW	≈ 107 GB/s	16%	472 GB/s	70%
Adam	BW	—	—	near-ceiling	—
Paged Decode	BW	—	—	435 GB/s	65%

Table 3: Per-kernel performance: baseline vs. optimized on RTX 6000 (sm_75). Peak: 16.31 TFLOPS FP32, 672 GB/s HBM. All numbers from Nsight Compute training-run profiles (1 step, 4-layer, $C=256$, $S=128$). Flash Attention at 10% of FP32 peak in both profiles because only 4–8 thread blocks launch against 72 SMs at this batch size; at full training shape it saturates the device. “—” indicates the kernel was not present in the before profile (Adam: launch-count window exhausted before the optimizer step; Paged Decode: added after baseline profiling). Adam reads/writes ≈ 20 bytes/param and is bandwidth-dominated by construction.

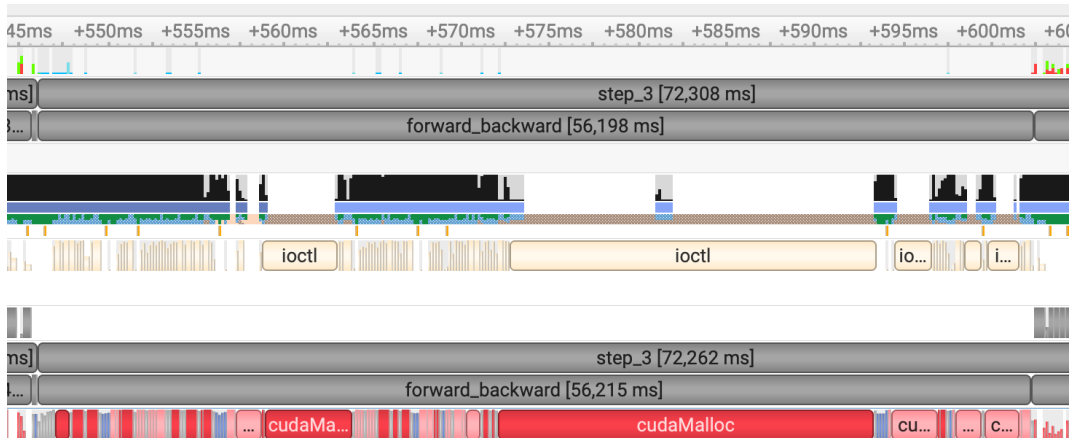


Figure 2: Nsight Systems timeline **before** pre-allocation: repeated `cudaMalloc` calls stall the kernel queue, producing ≈ 30 ms of idle GPU time per step.



Figure 3: Nsight Systems timeline **after** pre-allocation: the `cudaMalloc` stalls are absent and the kernel queue runs without interruption.

C Communication Benchmark

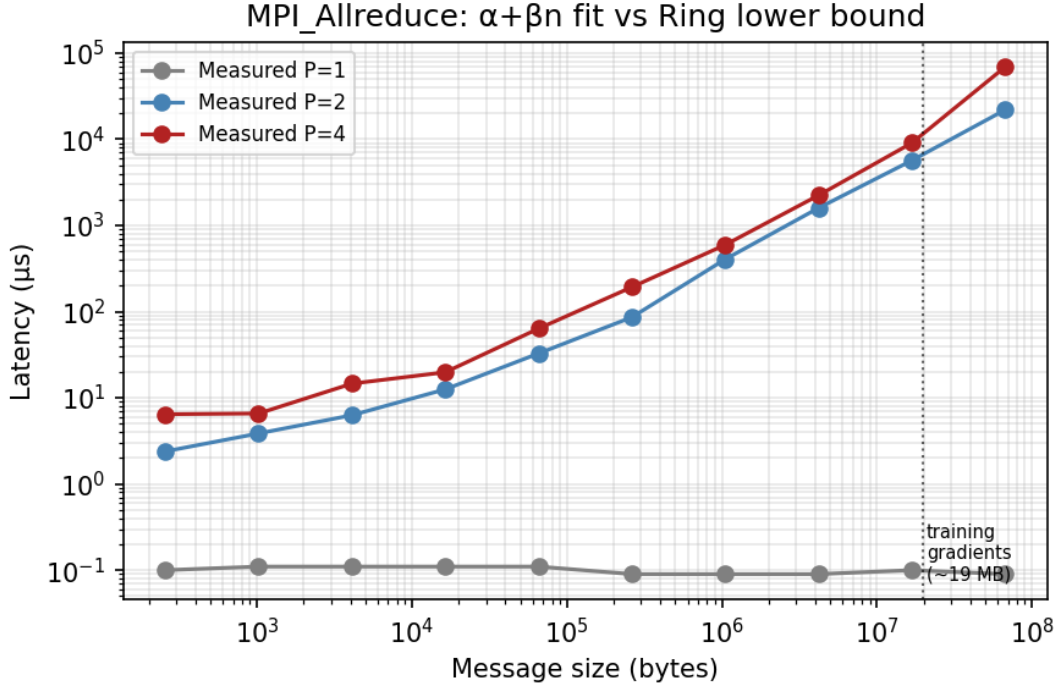


Figure 4: MPI_Allreduce latency vs. message size for $P=1$, $P=2$, and $P=4$, 100-trial averages. At small sizes latency is flat (α -dominated, startup cost per call); at large sizes it grows with slope 1 (βn -dominated), implying an effective bandwidth of ≈ 3 GB/s, constrained by the PCIe D→H→D round trip rather than the network. CUDA-aware MPI would bypass these copies.

D PagedAttention Benchmark

B	naive util	paged util	naive step (μs)	paged step (μs)	overhead
1	100.0%	100.0%	15.0	15.6	+4.0%
4	43.6%	87.1%	15.1	15.8	+4.6%
16	51.9%	92.2%	17.9	18.7	+4.5%
64	39.4%	88.0%	38.0	38.5	+1.3%
256	44.4%	89.3%	129.2	130.7	+1.2%

Table 4: PagedAttention decode benchmark on a single RTX 6000. Workload: B concurrent requests, lengths $\sim \text{Exp}(64)$ capped at 128 tokens, $H=8$, $D=64$, block size 16, 200-iteration averages. *Naive util*: useful KV / contiguous-padded allocation. *Paged util*: useful KV / block-pool allocation. Indirection cost (last column) is $\leq 5\%$ across the sweep and converges to $\sim 1\%$ at large B where the kernel saturates HBM bandwidth.

E Scaling

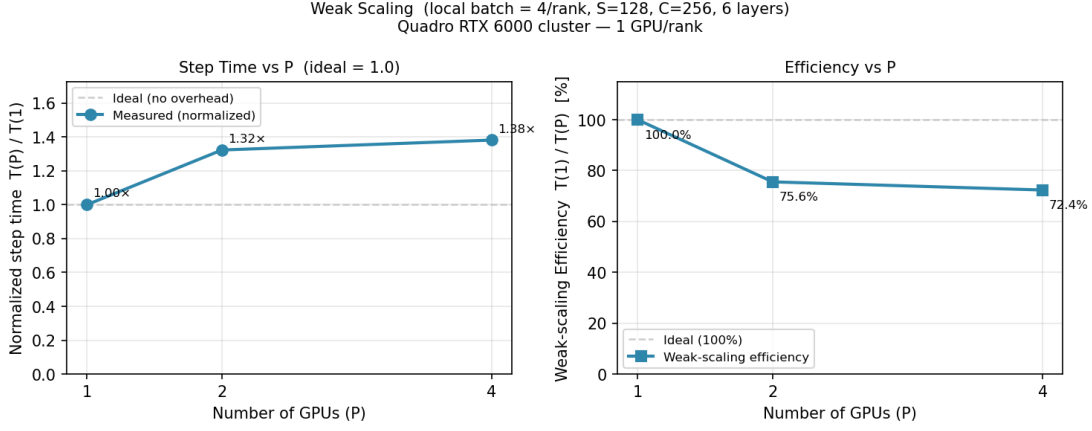


Figure 5: Weak scaling efficiency (local $B=4$ per rank). Efficiency at $P=4$ is 73%, driven by a nearly fixed comm cost (≈ 15.6 ms) regardless of batch size — confirming that comm cost is dominated by gradient size, not compute volume.

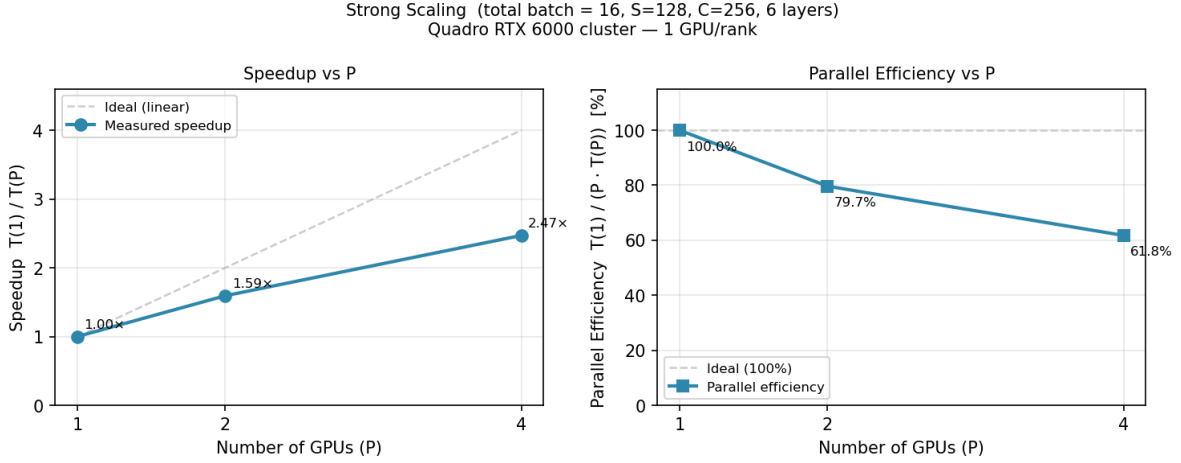


Figure 6: Strong scaling: step-time breakdown into compute (forward + backward + Adam) and communication (allreduce_gradients). Total $B=16$, 20-step averages.

F Kernel Correctness Details

Kernel	Dir	Worst-case config	Max abs	Max rel	Result
<i>Forward kernels (vs. CPU reference and PyTorch)</i>					
GEMM	Fwd	$M=256, N=256, K=64$	3.82×10^{-6}	$1.51 \times 10^{-3*}$	PASS
Flash Attention	Fwd	$B=2, H=4, S=64, D=64$	2.09×10^{-7}	$4.33 \times 10^{-3*}$	PASS
LayerNorm	Fwd	$N=128, H=768$	3.58×10^{-7}	$7.93 \times 10^{-3*}$	PASS
GELU	Fwd	$N=4096$	5.96×10^{-8}	1.87×10^{-7}	PASS
Softmax	Fwd	$N=512, V=30000$	8.00×10^{-11}	1.13×10^{-6}	PASS
Cross-Entropy	Fwd	$N=4096, V=30000$	1.91×10^{-6}	2.01×10^{-7}	PASS
<i>Backward kernels (vs. CPU reference)</i>					
LayerNorm (dx)	Bwd	$N=4096, H=768$	2.38×10^{-7}	$2.47 \times 10^{-2*}$	PASS
LayerNorm ($d\gamma, d\beta$)	Bwd	$N=4096, H=768$	1.53×10^{-4}	1.55×10^{-4}	PASS
GELU	Bwd	$n=100003$	1.19×10^{-7}	$5.54 \times 10^{-3*}$	PASS
Cross-Entropy	Bwd	$N=128, V=1024$	5.96×10^{-8}	1.38×10^{-6}	PASS
Attention (dQ, dK, dV)	Bwd	$B=2, H=4, S=64, D=64$	$\leq 10^{-5*}$	$1.68 \times 10^{-2*}$	PASS
Paged Attention	Inf	$B=64, D=64$	0	0	PASS

Table 5: Worst-case errors for each kernel across all test configurations. Pass condition: max rel $\leq 10^{-3}$ or max abs $\leq 10^{-5}$. *Relative error exceeds 10^{-3} ; the kernel passes via the absolute condition (abs $\leq 10^{-5}$), meaning large relative errors concentrate in near-zero elements. Attention backward reports relative errors only, abs $\leq 10^{-5}$ is inferred from the pass outcome. Paged Attention is verified bit-identically against the contiguous reference across all seven scrambled block-table configurations.